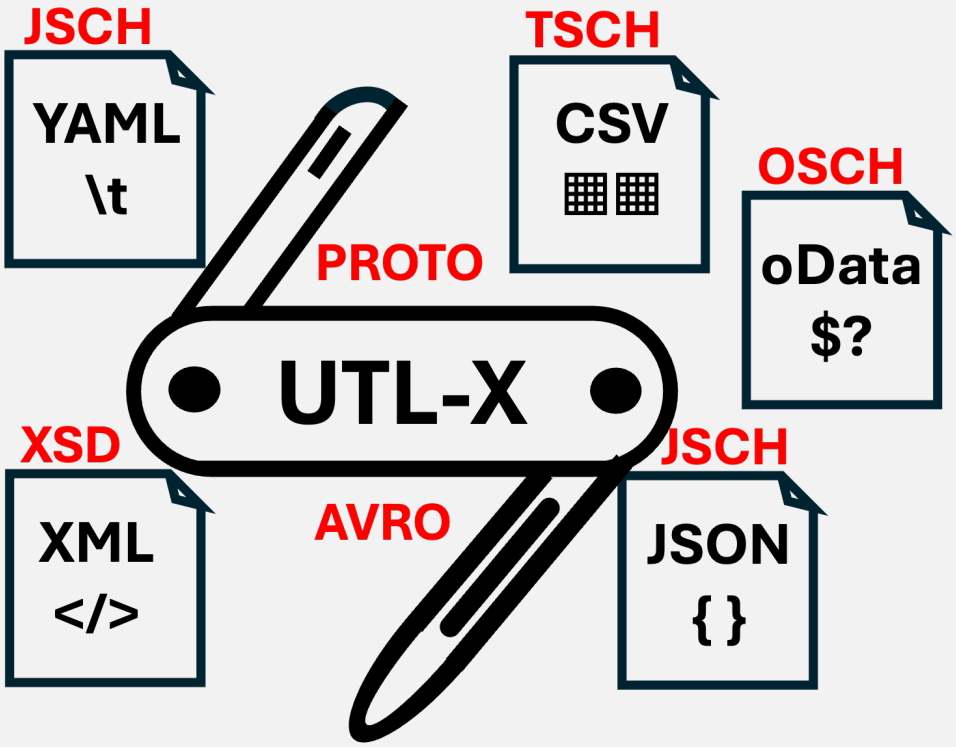
GLOMIDCO

UTLX

Universal Transformation Language eXtended



From Many, One

The Theory of N:1 Data Mapping — the Universal Data Model, Message Contracts, and Schema-Driven Mapping

Ir. Marcel A. Grauwen

First Edition — 2026

From Many, One: The Theory of N:1 Data Mapping — the Universal Data Model, Message Contracts, and Schema-Driven Mapping

Published by GLOMIDCO B.V., The Netherlands

ISBN to be assigned.

First edition, 2026.

UTL-X is open source software. The language specification, CLI tool, and standard library are freely available at `https://github.com/grauwen/utl-x`.

This is a companion volume to **UTL-X: One Language, All Formats**. Where that book teaches the language, this one develops the underlying theory of mapping many inputs to one output.

Typeset with Typst in New Computer Modern.

About the Author

Ir. Marcel A. Grauwen is a Dutch software engineer and architect with over twenty-five years of experience in enterprise integration, data transformation, and middleware platforms.

He has designed and built integration solutions on Tibco BusinessWorks, MuleSoft, SAP CPI, Azure Logic Apps, and IBM Integration Bus, and grew increasingly frustrated with the fundamental limitation they all shared: format-specific transformation logic tied to vendor platforms.

UTL-X was born from that frustration — one transformation language that works on all data formats, runs anywhere, and belongs to no vendor. Marcel designed the language, built the runtime (CLI, IDE daemon, and production engine), wrote the standard library, and created the USDL schema-classification system. This book grew out of the design work behind the Message Contract mode and the Open-M multi-input pipeline: the recurring question of **how a transformation that merges many inputs into one contract can be reasoned about, not just written.**

Marcel holds an Ir. degree (Master of Science in Engineering) and is based in the Netherlands.

Table of Contents

Preface

Why This Book

There is a companion to this volume, *UTL-X: One Language, All Formats*, that teaches you how to write transformations. This book is about something quieter and, in the long run, more important: how to **reason** about them.

The two books answer different questions. The first asks, "How do I transform this XML into that JSON?" This one asks, "What **is** a mapping — as an object I can analyse, score, and check for correctness — when several inputs must converge into one fixed output contract?"

That convergence — many inputs, one output — is the shape almost every real integration eventually takes. A purchase-order message arrives. It is not enough on its own: the line items carry product codes that must be expanded from a catalogue; the totals are not present in the source and must be computed; a handful of environment values — a tenant region, a correlation id — must be stamped in; and the whole thing must satisfy an invoice schema that someone else owns and will not change for your convenience. That is an **N:1 mapping**, and writing it by hand is a craft. Reasoning about it — knowing it is complete, knowing which fields are derived and which are copied, knowing where a human must still decide — is a theory.

The good news is that the theory already exists, scattered across four decades of database research: schema matching, schema mapping, data exchange, data provenance. None of it was written with UTL-X in mind. All of it applies. This book gathers that material, translates it out of its native dialect, and lands it on UTL-X's two concrete foundations — the **Universal Data Model** and the distinction between **Execution mode** and **Message Contract mode**.

What This Book Is, and Is Not

This is a **theory** book. It contains very little UTL-X syntax. Where the companion volume shows you `concat`, `map`, and `groupBy`, this book shows you **why** a given output field demands an aggregation, **why** a second input is a lookup rather than a co-equal source, and **why** the mapping you wrote is — or is not — sound.

It is not a reference manual, and it is not a tutorial. It is the book you read when you want to understand the machinery behind Message Contract mode: the schema graph, the correspondence set, the strategy pipeline, and the line — drawn as sharply as we can draw it — between what a deterministic algorithm can decide and what genuinely needs a human or an LLM.

Who Should Read It

Read this if you are an **integration architect** who has built a hundred mappings by feel and wants the vocabulary to say precisely what they do; a **platform engineer** designing tooling that must analyse mappings rather than merely run them; or a **researcher or student** who wants to see classical schema-mapping theory applied to a working, open transformation language rather than a paper prototype.

You do not need a database-theory background — every borrowed idea is introduced from first principles — but you should be comfortable with the notion of a schema, a tree, and a function.

How to Read It

The book is in three parts.

Part I — Foundations builds the ground the rest stands on: the N:1 problem itself, the Universal Data Model as a formal object, and the two-modes distinction (Execution versus Message Contract). Read it first; everything else assumes it.

Part II — The Theory of Mapping is the heart of the book. It treats schemas as graphs, mappings as sets of source-to-target dependencies, and the analysis of a mapping as a scored correspondence set. This is where the classical literature enters.

Part III — From Theory to Transformation turns the theory into a working pipeline: typing the inputs, analysing the outputs, inferring functions, and generating idiomatic UTL-X strategy-first.

A note on cross-references: throughout, design decisions are tagged with their origin in the UTL-X feature record (IF11 coverage, IF20 strategy, IF21 the visual mapper). You do not need those documents to read the book; the tags simply mark where theory met implementation.

Introduction: What Is Mapping?

Before a theory of N:1 mapping, a plainer question deserves an answer: what is **mapping** in the first place? The word is used loosely across the industry — for data conversion, for field assignment, for integration glue of every kind. This book needs it sharp. This introduction supplies the definition, the reasons mapping is harder than it looks, and a short account of how the discipline arrived where it is. Readers fluent in data transformation can skim it; everyone else should start here, because every later chapter assumes this vocabulary.

Mapping, Defined

A **mapping** relates two data structures: a **source** and a **target**. It is the specification of how each part of the target is obtained from the source — which target field copies which source field, which is computed, which is looked up, which is left to a default. A **transformation** is the executable realisation of a mapping: a function that, given a concrete source, produces a concrete target satisfying the mapping.

Three words carry weight in that definition. **Source** and **target** name the two ends; in this book the target is privileged — it is a fixed contract — and the source is, in general, several inputs at once. **Specification** signals that a mapping is a description, not yet a program: it says **what** must hold between source and target, and a transformation is one way to make it hold. Keeping the mapping (the **what**) distinct from the transformation (the **how**) is the move that lets a mapping be analysed before it is executed — the whole premise of Message Contract mode in Chapter 3.

The smallest mapping relates one source to one target: the **1:1** case. It is where the field's theory was developed, and it is the right place to build intuition. This book is about the larger case — many sources, one target, the **N:1** mapping of Chapter 1 — but everything in it rests on the 1:1 foundation, which Chapter 12 documents in full.

A Word on "Transformation"

The phrase "data transformation" means different things to different communities, and it is worth saying plainly which one this book is about. To a data scientist, transforming data means **feature engineering** — scaling, normalising, reshaping a table so a model can learn from it. To a business leader, "data transformation" may mean an organisational programme: new governance, new platforms, a change in how a company treats data as an asset. Neither is the subject here.

This book is about **structural and semantic data mapping**: taking data in one representation — a format, a schema, a contract — and producing it faithfully in another. It is the transformation of integration and interchange: an order becomes an invoice, an XML message becomes a JSON document, several inputs converge into one output that satisfies a fixed contract. The values are not being analysed or modelled; they are being **moved and reshaped** from one shape to another, correctly. When this book says "transformation," it means that — and only that.

Why Mapping Is Hard

If mapping were merely copying fields with matching names, there would be no theory and no book. It is hard because source and target disagree along three independent axes, and a real mapping must reconcile all three at once.

Format heterogeneity. The source may be XML and the target JSON; one has elements, attributes, and namespaces, the other objects, arrays, and a fixed set of scalar types. The same information wears different syntactic clothes. This is the most visible difficulty and, as it turns out, the most superficial — a common data model dissolves it, which is why Chapter 2 begins there.

Structural heterogeneity. Even within one format, source and target organise the same information differently: a flat list on one side, a nested header-and-detail tree on the other; a repeating group here, a joined reference table there. Reconciling structure is not syntactic translation; it is reshaping, and it is where the interesting mapping shapes — joins, groupings, flattenings — live.

Semantic heterogeneity. Hardest of all, the two sides may mean the same thing by different names, or different things by the same name. `customerId` and `clientRef`; two unrelated fields both called `id`. No amount of format normalisation or structural analysis closes a semantic gap; it requires knowledge of what the data **means**. This is the axis where deterministic methods reach their limit and human or machine judgement must enter — a boundary this book draws as sharply as it can.

A mapping is the act of reconciling all three. Much of the craft, and all of the theory, is about handling each with the right tool: a common model for format, a schema graph for structure, and a careful, explicit treatment of semantics that never pretends to more certainty than it has.

A Short History

Mapping has two lineages — one in practice, one in research — that ran in parallel for decades and only recently began to converge.

The **practitioner** lineage is a story of format-specific tools. XSLT, standardised in 1999, gave the XML world a powerful, declarative transformation language, and for a generation "data mapping" and "XSLT" were nearly synonyms — as long as the data was XML. As JSON rose, jq filled the same niche for it; YAML, CSV, and the rest each acquired their own utilities. Visual mappers — IBM's BizTalk Mapper, Altova MapForce, the graphical tools inside every ETL suite — made mapping approachable by drawing lines between source and target trees, with "functoids" in the middle for the computations, an idiom this book's tooling deliberately echoes. DataWeave, arriving with MuleSoft around the middle of the 2010s, did something genuinely new for the mainstream: it abstracted the format away, so one transformation worked across XML, JSON, and CSV alike. Its limitation was ownership, not idea: it lived inside a single vendor's platform.

The **research** lineage ran quietly alongside, in the database community, and it is the one this book draws on. Schema matching — automatically finding the correspondences between two schemas — was surveyed definitively by Rahm and Bernstein in 2001. Schema mapping — turning those correspondences into an executable relationship — was pioneered by the Clio project at IBM through the 2000s, which established that joins and nestings are derived from keys and foreign keys, not guessed from names. Data exchange gave the whole enterprise a formal object: a mapping as a set of source-to-target dependencies. And the provenance and data-classification communities supplied the vocabulary for **how** an output value is produced and **what kind** of data each input is. Chapter 12 is an annotated tour of this canon.

The two lineages converge here. The practitioner's visual mapper and the researcher's schema graph are the same object drawn differently; the functoid and the derived field are the same idea named differently. What has been missing is a treatment that takes the research seriously and lands it on a working, open, format-agnostic language — which is precisely the gap this book sets out to fill.

How to Think About Mapping

If there is one habit of mind to carry into the rest of the book, it is this: **a mapping is an object to be reasoned about, not merely a script to be written.** The practitioner tradition, for all its tools, has largely treated mapping as authorship — you sit down and write the assignments. The research tradition treats it as something with structure, completeness, and provenance — something you can analyse, score, and check. This book sides firmly with the second view, while keeping the first view's hard-won practicality. The reward is the ability to say, precisely, what a mapping **does** and whether it is **right** — before a single byte of real data has flowed.

With that, the subject proper can begin. Chapter 1 states the N:1 problem; Chapter 2 builds the common model that makes "all formats" mean something; Chapter 3 draws the line between reasoning about schemas and transforming data. Everything after is consequence.

Part I

Foundations

The problem, the data model, and the two modes

1 The N:1 Mapping Problem

Most writing about data transformation quietly assumes a tidy shape: one source document, one target document, a function between them. That assumption is comfortable and almost always wrong. Real integrations converge. Several inputs — of different kinds, playing different roles — must be folded into a single output that satisfies a contract you do not own. This chapter defines that problem precisely, because everything else in the book is an answer to it.

1.1 One Output, Many Inputs

Call it *N:1 mapping*: N inputs in, one output out. The output is the privileged party. It is fixed — a schema someone else published — and the entire job is to satisfy it. The inputs are the variables. They differ not just in format but in **role**: one carries the primary data, another is a reference table joined by key, a third is a set of environment constants injected at run time.

Consider a concrete case that runs through the whole book. An order-management system emits a purchase order. A downstream system requires an invoice. Between them:

- a **payload** — the purchase order itself, rich and nested: a header, customer details, and a repeating group of line items;
- a **lookup** — a product catalogue, used only to expand each line's product code into a name and description;
- a **prior step** — the output of an earlier pipeline stage that already validated and enriched the order (its approval status, say);
- **configuration** — a few name–value pairs: the billing region, a correlation id, a tenant identifier.

Four inputs. One invoice. The invoice's `lineItems` come from the payload's lines, but each line's `productName` comes from the catalogue; its `orderTotal` appears in **none** of the inputs and must be computed; its `status` comes from the prior step; its `billingRegion` is a constant. No single input is "the source." The output is assembled from all four, each contributing in a structurally different way.

1.2 Why N:1 Is Not Just 1:1 Repeated

The naive view is that an N:1 mapping is simply several 1:1 mappings stacked together — source each output field from whichever input happens to have it. That view fails for three reasons, and naming them frames the rest of the book.

The inputs are not symmetric. A 1:1 framing treats every input as an equal candidate source for every output field. But a lookup table is not a co-equal source; it is a dimension joined by a foreign key. Treating the catalogue as "just another place to find fields" misses that the relationship between payload and catalogue is a **join**, with a key, producing exactly one UTL-X construct — and the wrong framing will never reach for it.

Some outputs have no source at all. `orderTotal` is not present anywhere; it is a **derivation** — a sum over the line items. A field-matching view reports it as an unmatched gap or, worse, as matched to some unrelated numeric field. Neither is correct. The output field is real and producible; it simply requires computation, not copying.

The shape of the answer varies by sub-problem. Part of the mapping is a pass-through (copy the header), part is an array transform (map the lines), part is a join (expand product codes), part is an aggregation (sum the totals), part is a constant (stamp the region). A single uniform strategy — "fill every field from a source" — produces verbose, idiomatically poor transformations because it cannot bend to these different shapes.

These three failures — asymmetry, derivation, and shape — are the reason a **theory** is worth having. Each is addressed head-on in later chapters: input roles in Chapter 8, derivation and output analysis in Chapter 9, and mapping shapes in Chapter 11.

1.3 Open-M: The Concrete N:1 Model

UTL-X's production pipeline, *Open-M*, gives the abstract N:1 problem a fixed structure. An Open-M transformation receives a known set of input slots:

Slot	Role	Nature
`payload`	primary data, full structure	a schema — structurally rich
`payload-1 … payload-N`	prior pipeline-step outputs (chained)	a schema — structurally rich, with temporal order
`lookup`	reference / enrichment data, joined by key	a schema — structurally rich
global config	environment constants	name–value pairs — not a schema
shared variables	run-time variables	name–value pairs — not a schema
pipeline variables	six fixed named inputs, always present	name–value pairs — not a schema

This is the first hint of the structure that organises the whole theory. The slots fall into two camps. Some are **structurally rich** — they have nesting, keys, and foreign-key relationships, and they participate in genuine structural mapping. Others — the name–value pairs — carry no structure to align at all; they are constants injected at mapping time, closer to what a programming language calls parameters than to a source schema. Chapter 8 makes this split formal; for now it is enough to see that "the inputs" are not one homogeneous thing.

1.4 What Must Be True of the Output

The output's privilege — being fixed — is also a discipline. A correct N:1 mapping is not merely one that runs without error; it is one that **covers** the contract. Every required field of the output must be accounted for: copied from a source, derived by computation, looked up from a reference, supplied as a constant, or — honestly — flagged as a gap that no input can satisfy.

This gives us our first definition of correctness, which the rest of the book sharpens:

> A mapping is **schema-complete** when every required output field has a defined provenance — copy, derivation, lookup, constant, or an explicit, acknowledged gap.

Note what this definition does **not** say. It says nothing about real data. A mapping can be schema-complete and still fail on a particular instance — a lookup that finds no matching row, a number that overflows. That second kind of correctness is a different question, asked of real values rather than schemas, and it belongs to a different mode of the system entirely. Separating those two questions — completeness against the schema, adequacy against the data — is so important that the next chapters give each its own foundation: the Universal Data Model in Chapter 2, and the two modes in Chapter 3.

1.5 The Shape of the Answer

The remainder of the book builds, piece by piece, an answer to the N:1 problem:

- Represent every input and the output in **one** model, so that "all formats" is not a slogan but a fact (Chapter 2).
- Separate reasoning-about-schemas from transforming-data into two clean modes (Chapter 3).
- Treat each schema as a **graph** of entities, keys, and foreign keys (Chapter 4).
- Borrow the discipline of **schema matching** to find candidate correspondences (Chapter 5).
- Make the mapping a **formal object** — a set of source-to-target dependencies — that can be reasoned about (Chapter 6).

- Record the analysis as a scored **correspondence set** (Chapter 7).
- **Type** the inputs by role and the outputs by derivation (Chapters 8 and 9).
- **Infer** the UTL-X function each correspondence needs (Chapter 10).
- Generate the transformation **strategy-first**, choosing the shape before filling the fields (Chapter 11).

None of these steps is new to computer science. The contribution is in assembling them into a single, coherent account of how many inputs become one output — and in grounding that account on a data model and a mode distinction that actually ship.

2 The Universal Data Model

A theory of mapping needs a common ground for the things it maps. If a "mapping" had to be defined separately for XML-to-JSON, CSV-to-XML, YAML-to-OData, and every other pairing, there would be no theory — only a catalogue of special cases. The Universal Data Model (UDM) is that common ground. This chapter develops it as a formal object, because the rest of the book reasons over UDM and nothing else.

2.1 The Combinatorial Argument for a Common Model

Begin with the problem UDM solves. Suppose a system must transform between F formats directly, format to format. Each ordered pair needs its own conversion logic, with its own handling of that pair's quirks — attributes and namespaces for XML, headers and type inference for CSV, anchors and tags for YAML. The number of such paths grows as $F \times (F - 1)$: quadratic in the number of formats. Every new format multiplies the work.

Interpose a single neutral model and the quadratic collapses to linear. Each format needs one **parser** into the model and one **serializer** out of it — $2F$ pieces of logic, never F^2. More importantly for us, the transformation itself is written once, against the model, and is **format-blind**: it cannot tell, and need not care, whether a value arrived as a JSON number or an XML text node.

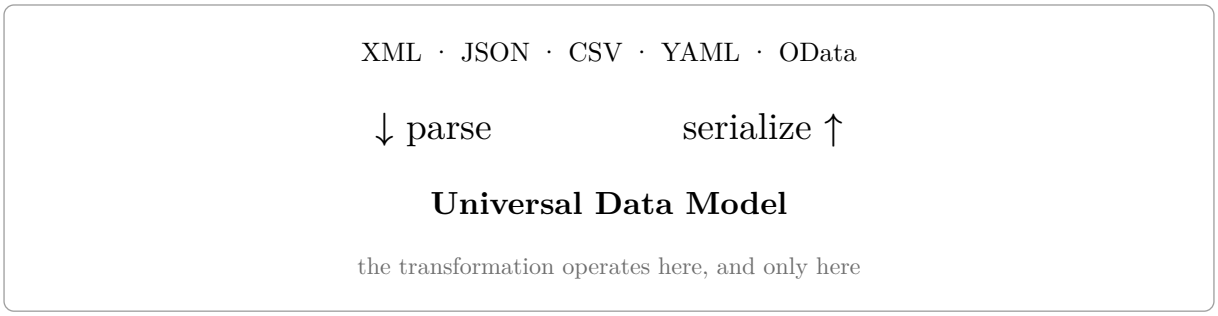


Figure 1: One model in the middle turns F^2 format-to-format paths into $2F$ parse/serialize pieces. The transformation never touches a raw format.

This is not a UTL-X invention; it is the **canonical data model** pattern from enterprise integration, and before that the pivot representation of any multi-format compiler. What matters for the theory is the consequence: a mapping is a function from UDM to UDM. Formats live only at the edges.

2.2 UDM as a Value Algebra

Formally, UDM is a recursively defined set of values. A UDM value is one of:

Constructor	Holds
Scalar	a single atomic value: string, number, boolean, or null
Object	a finite map from names to UDM values, with optional attributes and metadata
Array	an ordered sequence of UDM values
DateTime, Date, LocalDateTime, Time	first-class temporal values
Binary	a raw byte sequence

The first three constructors carry the entire structural story; the temporal and binary types are conveniences that spare every transformation from re-parsing dates and re-decoding bytes. There is also an internal Lambda value for function arguments to higher-order operations, but it never appears in data.

Two design choices in the Object constructor repay attention, because they are where the messy realities of formats are quarantined rather than leaked.

Attributes are separate from properties. An XML element's attributes are kept in their own map, distinct from its child elements. This preserves XML's element/attribute distinction without forcing it on formats that have no such notion — JSON simply never populates the attribute map. A transformation can treat attributes and properties uniformly when it wants to, or distinguish them when it must.

Metadata is a reserved, format-fidelity channel. Each Object may carry a small metadata map holding things a faithful round-trip needs but the data itself does not — an XML element's namespace context, or the detected XSD design pattern. This channel is dropped by the ordinary JSON, XML, and YAML serializers (the data is what survives) but **preserved** by the dedicated UDM-language serializer, so a UDM value can be written out and read back without losing format-specific fidelity. The lesson generalises: a clean data model needs a side-channel for the inconvenient truths of real formats, kept strictly apart from the data.

2.3 Shape and Data: One Model, Two Readings

Here the theory makes a distinction that the rest of the book leans on constantly. The same UDM tree can be read in two ways.

Read for its **values**, a UDM tree is an **instance** — concrete data, the parsed content of one document. Read for its **shape** — the names, the nesting, the types, the cardinalities, with the particular values abstracted away — it describes a **schema**.

This is why UDM can represent both the data flowing through a transformation and the schemas a mapping is reasoned about. A node is the same kind of node in both readings; what differs is whether you care about the value at a leaf or only its type and obligations (required, optional, repeating). In the UTL-X tooling this appears as a single node type for both: the schema-field node is literally an extension of the data-field node, adding only schema-specific annotations such as "required" and "constraint." Shape is data with the values forgotten and the obligations remembered.

Keeping this distinction explicit pays off immediately. A **schema graph** — the object Part II reasons over — is the shape reading of UDM, enriched with keys and foreign keys. A **data UDM** — the object a transformation actually produces — is the value reading. They share a coordinate system (the same paths address both), which is what lets a tool align an instance onto its schema without either holding a pointer to the other. We will use that alignment in Chapter 3 and again when we visualise mappings.

2.4 Schema Is Metadata

The shape reading has a more familiar name: **metadata** — data about data. A schema is exactly that: a formal description of a **class** of data instances — their names, types, cardinalities, and constraints — with no particular instance's values. To say UDM has a shape reading and a data reading is to say it can be read as metadata or as data: the schema graph of Part II is metadata; the instance a transformation produces is data; and they share one node model because metadata is, in the end, data with the values abstracted and the obligations retained.

This invites a ladder. Data is described by metadata — a schema. Metadata is in turn described by **meta-metadata** — a schema of schemas, a registry. The `meta-schema` input type of Chapter 8 and the schema-document envelope that records a schema's own type both sit one level up: they are data about the metadata.

One caution, because "metadata" is as overloaded as "transformation." The sense above — a schema is metadata about a **class** of data — is distinct from the small format-fidelity **metadata channel** a UDM object carries (a namespace, a detected XSD pattern), which is metadata about **one value's representation**, a narrower thing; and from the enterprise `metadata` data class of Chapter 8 (catalogues, lineage), which is data about an organisation's data assets. All three are honestly "data about data," but at different scopes — class, value, and asset. When this book calls a schema metadata, it means the first.

2.5 Why the Model Must Carry Its Own Meaning

A subtle but load-bearing principle governs UDM's design: **the model should carry everything a downstream consumer needs, so that no consumer has to reach back to the original format.**

When a schema is parsed, the format-specific facts that matter — an element is a key, this reference is a foreign key, that group repeats — must be lifted into the UDM representation as first-class annotations. If they are left behind in the XSD or JSON-Schema text, then every later stage (matching, strategy selection, code generation) must re-open the source format and re-derive them, and the model has failed at its one job. The matchers and strategies in Part II are therefore defined over UDM schema graphs, never over raw XSD or JSON-Schema syntax trees. There must be one schema graph, not four format-specific ones.

This principle also tells us where **not** to put things. Schema-level facts — what kind of input this is, how it is classified — belong to the schema reading, attached to the schema object as a whole, never smeared across the data nodes of an instance. The data model represents values; it does not represent a value's role in some larger pipeline. We return to this boundary in Chapter 8, where inputs are typed: the type is a property of the **schema document**, not of any datum inside it.

2.6 What UDM Buys the Theory

With UDM in place, the central objects of the book can be stated cleanly:

- An **input** is a UDM value (its data reading) described by a UDM schema graph (its shape reading).
- The **output contract** is a UDM schema graph — pure shape, fixed in advance.
- A **transformation** is a function from a tuple of input UDM values to one output UDM value.
- A **mapping** — the thing we analyse — is a relationship between the input schema graphs and the output schema graph, independent of any particular data.

That last line is the quiet payoff. Because UDM separates shape from data, a mapping can be reasoned about entirely in the shape reading — before any instance exists. Whether to do so, and what becomes possible when we do, is the subject of the next chapter.

3 Two Modes: Execution and Message Contract

The previous chapter ended on a claim with large consequences: because UDM separates shape from data, a mapping can be reasoned about before any data exists. UTL-X turns that claim into an architectural fact by running in two distinct modes. *Execution mode* transforms real data. *Message Contract mode* reasons about schemas. They are not two views of one engine; they are two analyses with different inputs, different questions, and different notions of correctness. Conflating them is the single most common way to get confident, wrong answers about a mapping. This chapter keeps them apart.

3.1 The Distinction in One Sentence

> **Execution mode** asks: *given this concrete input, what output does the transformation produce?*
> **Message Contract mode** asks: *given these schemas, is a correct mapping structurally possible, and what does it consist of?*

Execution mode operates on the **value** reading of UDM (Chapter 2): instances, real bytes, actual line items. Message Contract mode operates on the **shape** reading: schema graphs, with values abstracted away. One runs the function; the other studies it.

3.2 Why the Two Cannot Be Merged

It is tempting to treat Message Contract mode as "Execution mode without data" — run the transform on a sample and inspect the result. That shortcut is wrong, and the reason is foundational, not incidental.

A mapping can be **schema-complete but instance-incomplete**: structurally every output field has a source, yet on a particular input a lookup finds no matching row and a required field comes out empty. And a mapping can be **instance-adequate but schema-unsound**: it happens to produce valid output for the three samples you tried, while violating the contract for inputs you have not seen. These two failure modes are independent. Neither analysis can detect the other's failures, because they are asking different questions of different objects.

This is not a UTL-X peculiarity. It is the oldest distinction in the schema-matching literature — Rahm and Bernstein's *schema-only versus instance-based* axis — and it is older still in logic, as the difference between a property that holds by form and one that holds in a particular model. The practical upshot is firm: Message Contract analysis must be **schema-based**, computed from the schema graphs and their constraints; it must not smuggle in evidence from a sample instance and call the result a proof.

3.3 What Each Mode Sees

The two modes do not merely ask different questions; they are handed different things to begin with.

	Execution mode	**Message Contract mode**
Reading of UDM	value (instance)	shape (schema graph)
Inputs present	real data per message	the input schemas; sample instances are optional
The output	a produced instance	the output contract (a schema); no instance is produced
Runs the transform?	yes — real execution	no — analysis only
Correctness asked	adequacy on this data	completeness against the contract
Cost model	per message, on the hot path	once, when schemas load or change

Two rows deserve emphasis because they are routinely misread.

Message Contract mode is not "no instances." An input in Message Contract mode commonly carries a **sample** instance alongside its schema — a representative document, useful for preview and for inferring structure when a formal schema is thin. What Message Contract mode lacks is **execution**: it does not run the transformation to produce output data. The presence of sample data and the act of executing are different things, and only the latter is absent. A sample is evidence about shape; it is never the basis of a completeness claim.

The output sides differ sharply. In Execution mode the output is a produced instance — the result of running the function. In Message Contract mode there is no produced output instance, only the output **contract**. If a sample output document is wanted, it should be **synthesised from the contract** — generated from the schema — not obtained by executing the transform. The instinct to "just execute to see the output" quietly crosses the mode boundary and imports instance evidence into a schema question.

3.4 Three Times: Design, Init, and Run

The two modes line up with the phases of a system's life — but the run-time side has **two** phases, not one, and naming the middle phase repays the effort. There are two modes, and three times.

Message Contract mode is **design time**. Its analyses — classifying inputs, building schema graphs, finding correspondences, reporting coverage — happen while a developer authors or validates a mapping, when schemas are loaded or changed. They may be expensive; they run rarely, and never while messages flow.

Execution mode spans the other two times. **Init time** is the moment a transformation is deployed and loaded: the engine compiles the script once, establishes the pipeline's input slots, binds the fixed configuration constants, and loads and indexes any static lookup tables — all before a single message arrives. **Run time** is per message, on the hot path: parse the input data into the model, execute, serialise.

Time	When	Work done
design	authoring / validation	typing, matching, the correspondence set — computed and recorded
init	deploy / load, once	compile the script, set up slots, bind config, index static lookups
run	per message	parse data into the model, execute, serialise — the hot path

Two consequences make the third phase worth its name.

It grounds the `config` / `nvp` split. Chapter 8 attributes the difference between configuration and run-time variables to **lifecycle** rather than structure — and the init/run boundary is exactly that lifecycle. A `config` value is fixed at init time and may be inlined into the compiled transformation; an `nvp` value varies per message and is read at run time. Identical flat shape; different **time**. The temporal model is what makes "lifecycle, not structure" precise.

It sharpens the hot-path discipline into three verbs. "No schema analysis on the hot path" becomes: a schema fact is **computed** at design time, **loaded** at init time, and only **used** at run time. The per-message path re-derives nothing, because the work was done earlier and recorded. When Part III folds input typing "into parsing," it means the **schema** parse — a design-time act, repeated at most at init when a bundle loads — never the per-message data parse that feeds Execution.

A caution to close: init time is **not** a third mode. There remain two modes — reason about schemas, or transform data. Init time is the operational seam **inside** the Execution side, between "set up once" and "run per message." Two modes, three times.

3.5 A Shared Coordinate System, Not a Shared Object

If the modes are so separate, how do they relate at all? Through coordinates, not references.

A schema graph (shape) and an instance (data) addressed by the **same paths**. The output contract's `lineItems[*].productName` names a position; an instance's value at that position is found by the same

path. This shared coordinate system lets a tool **project** an instance onto its schema on demand — to preview values against the contract, or to colour a schema node by whether a sample populated it — without either object holding a pointer to the other. The relationship between a schema and its instances is "conforms to," established per operation, not an ownership link baked into the data.

This matters for N:1 mapping specifically. In Message Contract mode the analysis lives entirely in the shape reading: input schema graphs on one side, the output contract on the other, correspondences between them. Sample instances, when present, are projected onto those graphs for human insight, but they never define the correspondences. The mapping is a relationship between **shapes**; data is how you check it later, in the other mode.

3.6 Where This Book Stands

Almost everything that follows is Message Contract mode. Part II's schema graphs, matchers, and correspondence sets are design-time objects in the shape reading. Execution mode appears only at the edges: as the thing whose correctness Message Contract mode cannot fully guarantee (instance-adequacy is its province), and as the run-time path that must be kept pristine.

Holding the two modes apart is the discipline that makes an N:1 mapping **analysable** at all. It lets us ask, and answer, "is this mapping complete?" without running it — and to know exactly what that answer does, and does not, promise about the day real data arrives.

Part II

The Theory of Mapping

Schemas as graphs, matching, the formal mapping object, and the correspondence set

4 Schemas as Graphs

The strategy of N:1 mapping rests on one move, and this chapter is about making it: stop reading a schema as text and start reading it as a graph. A schema written in XSD or JSON Schema is a document with names, and names are the most seductive — and least reliable — signal a mapper can use. The real information is in the topology: which nodes are entities, which fields identify them, which references reach from one input into another. That topology is a graph, and the whole of Part II reasons over it.

4.1 Why Names Are the Wrong Substrate

A field called `customerId` in one schema and `clientRef` in another almost certainly mean the same thing; two fields both called `id`, in different entities, almost certainly do not. Name similarity is noisy in exactly the places that matter most: foreign-key joins, flattened repeating groups, and fields that share a generic name across unrelated entities. A mapper that works on names as surface syntax produces its worst errors precisely at the junctions where an integration's value lives.

The constraint-based signal is different in kind. A primary key, a foreign key, a cardinality, a containment — these are **structural facts**, declared in the schema and not subject to the vagaries of naming convention, abbreviation, or language. Following the schema-matching literature, the theory elevates the constraint-based signal above the linguistic one. Names become a tie-breaker among structurally plausible candidates, never a way to promote a structurally impossible one. To do that, the structure has to be a first-class object. That object is the schema graph.

4.2 One Graph, Built on the Universal Data Model

There is an immediate temptation to build a graph per format — an XSD graph, a JSON-Schema graph, an EDMX graph — each with its own notion of "key" and "reference." That way lies four diverging implementations and four sets of bugs. The model of Chapter 2 forbids it. Schema parsing lifts every format into the **shape** reading of UDM, and the schema graph is built on that single representation. `xs:key`, JSON-Schema `$ref` chains, and EDMX navigation properties are all read once, into one vocabulary of nodes and edges. The matchers of the next chapters are defined over UDM schema graphs, never over raw format syntax trees.

This is the principle from Chapter 2 — the model must carry its own meaning — applied to structure. Whatever a format declares about identity and reference is lifted into the graph at parse time as first-class annotations, so that no later stage must re-open the original XSD to recover them.

4.3 The Node Typology

Every node in a schema graph carries a structural role, derived from the graph and its constraints rather than from the node's name:

Node role	Meaning
entity / collection	a structure, possibly repeating; cardinality distinguishes array from object
key	an identifying field of an entity
reference (foreign key)	a field whose value identifies a node in another entity — a join edge
containment	a parent–child nesting relationship
value	a leaf scalar

This typology is what replaces the older, weaker vocabulary of "driver," "lookup," and "enrichment" inputs. Those roles are not primitive; they fall out of the graph. A lookup table is simply an entity that the primary data **references by foreign key**. The driver is the entity that the output's top-level collection iterates. Roles are derived from the schema graph plus its foreign-key edges — deterministic, and far stronger than any name heuristic.

4.4 Keys and Foreign Keys as Edges

Two kinds of edge carry most of the signal.

A **containment** edge is the parent–child relationship of nesting: an order contains its line items. Containment is cardinal — one-to-one for a singleton child, one-to-many for a repeating group — and the cardinality is itself a structural fact, read from `maxOccurs`, from a JSON array, or from a collection-typed navigation property.

A **reference** edge is a foreign key: a field whose **value** is the identity of a node elsewhere. A line item's `productCode` is not a description; it is a pointer into the product catalogue. The reference edge is the formal counterpart of the intuition that "the second input is a lookup." Where a reference edge crosses from one input's graph into another's, it is exactly a join; where it stays inside one graph, it ties an entity to its parent.

4.5 Cardinality and Weak Entities

Cardinality turns the graph from a static picture into a predictor of mapping shape. The pattern that recurs in almost every business message is **header–detail**: a parent entity and a repeating child collection bound to it. An order and its lines; an invoice and its line items; a shipment and its parcels.

In entity-relationship terms, the child of a header–detail pair is a **weak entity** under an identifying relationship: an order line has no independent existence: it is identified only in the context of its order. This is not a curiosity of terminology. The header–detail shape is exactly what predicts a `map` over the collection, and a `groupBy` when the detail must be rolled up. Reading cardinality off the graph is reading the strategy off the graph.

4.6 Intra-Document Structure: The IDOC

It is easy to assume that foreign keys only matter **between** inputs — the payload referencing a lookup. They matter just as much **within** one input, and missing this is missing most of the granularity.

Consider an SAP IDOC carried as XML. It contains a header segment with an order identifier and a set of line-item segments, each with a line identifier and a back-reference to its header. Orders and order-lines that point to each other are a node-level foreign-key relationship **inside the payload graph** — a header–detail pattern, not a new kind of input. It is exactly this relationship that predicts the mapping shape: a parent with a referenced child collection becomes a `map`, and a roll-up over that collection becomes a `groupBy`.

This is the level at which the granular "kind of message" lives. The input as a whole has a **role** — it is the payload. Its internal richness — entities, keys, the header–detail edges between them — is the **graph**, and the graph is where strategy is decided. A theory that types only whole inputs and never looks inside them cannot tell an order from an order-with-lines, and so cannot tell a copy from a `map`.

4.7 Reachability: Clio's Observation

The graph earns its keep through a single, powerful idea, due to the Clio line of work on schema mapping: **a mapping is graph reachability under integrity constraints.** Candidate correspondences between a source and a target are not guessed from names; they are **derived** from the paths the schema graph permits. A foreign-key-reachable path from a source node to a target node corresponds directly to a join, or a lookup, in the generated transformation. Joins and nestings are consequences of the keys and foreign keys in the graph, not independently authored decisions.

This reframes the mapper's job. It is not inventing a mapping and hoping it is consistent with the schemas; it is **reading off** the mappings the schema topology already supports, and scoring them. The schema graph is not a hint to a matching algorithm — it is the space the algorithm searches. The next chapter is about how that search is scored.

5 Schema Matching

The previous chapter turned schemas into graphs and observed that candidate mappings are paths the graph permits. But a permitted path is not yet a chosen one: of the many ways a source node **could** correspond to a target node, which actually do? Deciding that is the problem of *schema matching*, and it has a four-decade literature that applies almost without modification. This chapter borrows that discipline and lands it on UDM schema graphs.

5.1 The Rahm–Bernstein Taxonomy

The organising work is Rahm and Bernstein's survey of automatic schema matching, which classifies every matcher along orthogonal axes. Three of them structure this entire book.

Axis	**The two poles**
granularity	element-level — matching individual fields — versus structure-level — matching subtrees
evidence	schema-only — names, types, constraints — versus instance-based — actual values
signal	linguistic — names — versus constraint-based — keys, foreign keys, cardinalities, types

Each axis has already appeared in this book, which is the point: the classical taxonomy is not an analogy for UTL-X's design but its skeleton.

The **granularity** axis is the element/structure split that motivated Chapter 4: a credible matcher must work at the structure level, which is why the schema graph must precede element comparison. The **evidence** axis is the Execution-versus-Message-Contract distinction of Chapter 3: Message Contract analysis is schema-only and constraint-based; instance evidence belongs to the other mode. The **signal** axis is the names-versus-structure judgement of Chapter 4: constraint-based signals outrank linguistic ones. Reading the taxonomy is reading the architecture.

5.2 Element-Level Signals and Their Ceiling

At the element level, three signals compare a single source field to a single target field.

Name similarity normalises and compares identifiers. String distance — edit distance, token overlap — handles the easy cases (`firstName` versus `first_name`). Synonym dictionaries stretch a little further (`firstName` versus `givenName`). But there is a hard ceiling. No string metric and no fixed dictionary will bridge `firstName` to `name-given-at-birth`, or `customerId` to `clientRef`, or a Dutch field name to its English counterpart. These require world knowledge — the kind a human supplies instantly and a language model supplies reliably, and no deterministic linguistic rule supplies at all. Element-level name matching is therefore a **pre-processing** step and an **AI** step, with very little useful middle.

Type compatibility asks whether the source value could inhabit the target field: a string into a string, an integer into a number, a date into a date — with a normalisation step that collapses the many spellings of "number" and "date-as-string." Type compatibility rarely confirms a match on its own, but it frequently **refutes** one, and refutation is valuable: it prunes the space the structural matcher must search.

Value domain, where a sample is available, compares the ranges and enumerations the two fields draw from. In Message Contract mode this is used cautiously — it is instance evidence, and Chapter 3 warned against letting it masquerade as schema proof — but as a tie-breaker it is legitimate.

5.3 Structure-Level Signals

Structure-level matching asks the larger question: can this source **subtree** satisfy this target **subtree**? Its signals come from the graph.

Cardinality must agree, or the mismatch must be explained: a one-to-many source feeding a one-to-one target is a flattening or an aggregation, not a copy. **Nesting depth** and shape similarity favour correspondences that preserve structure over those that contort it. **Foreign-key reachability** — the

Clio signal — is the strongest of all: if a target subtree is reachable from a source subtree along key and foreign-key edges, the correspondence is structurally grounded, whatever the names say.

The decisive property of structure-level signals is that they **outrank** names. A repeating group of line items on the source and a repeating group of invoice lines on the target, both reachable by foreign key to a product entity, are a strong correspondence even when their field names share almost nothing. The graph knows what the dictionary cannot.

5.4 Composite Matchers

No single signal is sufficient, and the literature's answer is to **combine** them, which the theory adopts wholesale.

Cupid is structure-aware: it computes similarity and then propagates it up and down the schema tree, so that agreement among a node's children reinforces the node, and vice versa. **COMA** and its successor COMA++ make the combination explicit and configurable: several individual matchers, each producing a score, combined under tunable weights. This is the direct model for the scored correspondence set of Chapter 7 — many signals, a weighted composite, and the sub-scores kept visible. **Similarity Flooding** casts the problem as graph propagation: similarity flows along the edges of a combined graph until it settles, a method that maps naturally onto UDM schema-graph alignment.

The shared lesson is that matching is **multi-signal** and the combination must be **legible**. A matcher that emits only a final number is untuneable and unexplainable. One that emits a structural score, a type score, a name score, and a foreign-key flag can be reasoned about, corrected, and trusted in proportion to which signal carried it.

5.5 Structure First, Names as Tie-Breaker

The discipline that falls out of all this is a strict ordering, and it is the rule the whole pipeline obeys: **derive candidate correspondences from structure; apply name similarity only to choose among structurally compatible candidates; never let a name promote a structurally incompatible one.**

The reason is asymmetry of failure. A false structural match is rare and usually visible — the cardinalities or the foreign keys give it away. A false **name** match is common and invisible: two unrelated `id` fields, two `amount` fields in different entities. Letting names lead means importing the most frequent and least detectable error first. Letting structure lead, with names breaking ties, imports the rarest error and uses the most fallible signal only where the safe ones have already agreed.

5.6 Where Matching Ends

Schema matching is mostly deterministic, and that is its strength — the same graphs yield the same correspondences, every time. But it has a boundary, and honesty about that boundary is what keeps the analysis trustworthy. Implicit foreign keys (a `customerId` with no declared `keyref`), choice groups (which branch of an `xs:choice` is the real target?), and the semantic name gaps the element level cannot close — these resist any deterministic rule. They are not failures of the method; they are the precise points where world knowledge is required, to be supplied by a human or a language model and recorded as such. Where that boundary lies, phase by phase, and how the analysis marks which side of it each correspondence came from, is the subject of the chapters on the correspondence set and on function inference.

6 The Mapping as a Formal Object

So far a mapping has been a collection of correspondences — pairings of source nodes to target nodes, found in the graph and scored. That is enough to **describe** a mapping but not to **reason** about one. To ask whether a mapping is complete, whether it is sound, whether it produces a minimal output, the mapping itself must be a formal object with a definition. The data-exchange literature supplies exactly such an object, and this chapter introduces it — together with a clear statement of where UTL-X transformations outrun it.

6.1 A Schema Mapping Is a Triple

In data-exchange theory a schema mapping is written

$$M = (S, T, \Sigma)$$

where S is the source schema, T is the target schema, and Σ is a set of logical constraints that relate them. The mapping is not the data and not the transformation code; it is this triple — a specification of **what must hold** between source and target, against which any concrete transformation can be checked.

The N:1 case, which a casual reading of the matching literature would think exotic, is already contained here. Nothing in the definition requires S to be a single schema. In the relational setting S is a **set** of relations to begin with, and the constraints in Σ range freely over all of them. For UTL-X this means S is simply the union of the input schema graphs; the dependencies still span source to target. The many-inputs problem does not break the formalism — it is the formalism's default, briefly disguised by a tradition of drawing one source box.

6.2 Source-to-Target Dependencies

The constraints in Σ are **source-to-target tuple-generating dependencies** — tgds. A tgd has the shape

$$\forall x : P(x) \rightarrow \exists y : Q(x, y)$$

read as: for every tuple in the source matching pattern P, there must exist a tuple in the target matching Q. The value the target receives is built from x by projection and renaming — moving a value, possibly to a new name, possibly into a new structural position.

This is the formal core of "joins and nestings come from keys, not names." A tgd whose source pattern follows a foreign-key path expresses precisely a join; a tgd whose target pattern nests one entity inside another expresses precisely a structural reshaping. The Clio observation of Chapter 4 — that mappings are reachability under constraints — is the statement that the right tgds are **derivable from the schema graph**, not authored independently of it.

6.3 Value Invention: The Boundary of Copying

A tgd's existential, $\exists y$, hides a deep point. Some target values are projections of source values; others are **invented** — they exist in the target but correspond to nothing in the source. Data-exchange theory models invented values with **labelled nulls** and **Skolem functions**: placeholders, parameterised by the source tuple, standing for "a value that must exist here but is not copied from there."

This is the formal line between **copying** and **deriving**, and it is the spine of the output analysis to come. A target field filled by projection is a copy. A target field that is a labelled null — a value that must exist but is not present in any source — is a derivation: it must be computed, looked up, generated, or supplied as a constant. The existential quantifier is the theory's quiet admission that not everything in the output comes from the input, which is exactly the situation an invoice's `orderTotal` puts us in.

6.4 Three Kinds of Dependency for N Inputs

The Open-M inputs of Chapter 1 are not symmetric, and the formal object must reflect that. The tgds fall into three classes.

Class	Form and meaning
primary	$S_{\text{payload}} \to T$ — the core structural mapping; full schema-graph treatment applies
enrichment	$S_{\text{prior-step}} \to T$ — augmenting or overriding from a previous pipeline step; same structural class, with temporal provenance
lookup binding	$S_{\text{config/vars}} \to T$ — constants injected as values; no structural alignment, no scoring

The third class earns its separation. A name–value input — a configuration set, a bag of pipeline variables — carries no structure to align: no entities, no keys, no foreign keys. It is not a source schema in the matching sense at all; it is what data-exchange theory would call a set of **constants** and what a programming language would call parameters. Running structural matching over it is meaningless. Such inputs are recorded as **bindings** — direct value injections — not as scored correspondences. The formal object keeps them in a separate compartment, and so will the artifact of Chapter 7.

6.5 Where UTL-X Exceeds the Theory

It would be convenient if every UTL-X transformation were expressible as a set of tgds. It is not, and the gap is precisely the interesting part. A tgd moves and reshapes values; by explicit design in the data-exchange literature, it does **not** compute, aggregate, or invent values by calculation. Lay the UTL-X function space against that boundary:

Function class	Within tgds?
field reference, `pick`, `omit`	yes — projection
`...spread` / merge	yes — a tgd over several source relations
`map`, `mapBy` over a collection	yes — a tgd over a sequence
`findBy`, `filterBy` (keyed lookup)	partly — a foreign-key join, the Clio pattern
`filter`	partly — a selection predicate in the source pattern
`groupBy`	no — grouping has no tgd equivalent
`sumBy`, `countBy`, `reduce`	no — aggregation is beyond tgds
derived fields (`total = sum(qty * price)`)	no — computation is beyond tgds
`orderBy`	no — ordering is outside relational semantics

The tgds cover the structural skeleton of a mapping — the copies, the joins, the reshapings, the iterations. Everything below the line — grouping, aggregation, derivation, ordering — lies outside what the schema-matching and data-exchange literature models. These are not gaps in UTL-X; they are gaps in the **theory**, and naming them is what lets the analysis be honest. A target field that needs a `sum` is, formally, a labelled null the tgds cannot fill: a **derivation gap**. The next chapters give those gaps a first-class place in the analysis rather than letting them hide as false matches.

6.6 What the Formal Object Buys

Treating a mapping as (S, T, Σ) rather than as an ad-hoc list of assignments is not formalism for its own sake. It buys three things the rest of the book spends.

It buys a **definition of completeness**: every required target obligation is discharged by some dependency or honestly marked as an unfillable gap — the schema-completeness of Chapter 1, now precise. It buys a **separation of structure from computation**: the tgd-expressible skeleton is exactly the part a deterministic algorithm can derive from the graph, and the part beyond it is exactly where derivation, and human or AI judgement, must enter. And it buys an **account of the N inputs**: primary, enrichment, and binding are not informal labels but distinct classes of dependency, with distinct treatment. With the object defined, Chapter 7 can finally write the analysis down.

7 The Correspondence Set

A mapping that exists only in an analyst's head, or only as a generated transformation, cannot be reviewed, diffed, tuned, or trusted. The analysis of Chapters 4 through 6 has to be **written down** — in a form that a human can audit, a version-control system can track, and a machine can consume. That artifact is the *correspondence set*. This chapter specifies it, and in doing so makes concrete everything the formal object of Chapter 6 only defined.

7.1 From Coverage Report to Matrix

The simplest version of the analysis answers a one-input question: for each output field, is it covered by the single source, and if not, where is the gap? Generalise this to N inputs and it becomes a **matrix** — inputs across the top, output leaves down the side, each cell recording how that input contributes to that output field, if at all.

The matrix is the analysis artifact, and it does three jobs at once. It **reveals input roles**: a column touched only as a key is a lookup; a column feeding most leaves is the driver. It **drives strategy selection**: a high proportion of one-to-one mirror cells calls for spread; a key column calls for a join; an array driver calls for `map`. And it **grounds field-level sourcing**: within whatever strategy is chosen, each output field still knows where its value comes from. The correspondence set is this matrix, made precise and given scores.

7.2 Scores, and Why Sub-Scores Are Mandatory

Each correspondence carries a confidence. The naive choice is a single composite number, and it is a trap. A bare 0.87 is opaque: it cannot be tuned, because there is no knob; it cannot be explained, because there is no reason; and it cannot be trusted in proportion to its basis, because the basis is invisible.

The discipline, taken from the composite matchers of Chapter 5, is to **store the sub-scores beside the composite**. A correspondence does not record 0.87; it records that the structural signal was 0.91, the type signal 0.95, the name signal 0.61, and that the match was foreign-key reachable. Now the number is legible. A high composite carried by structure and foreign-key reachability is trustworthy; the same composite carried mostly by a name match is a candidate for review. The sub-scores tell you not just **how** confident to be but **why**, and which signal to distrust if the mapping later proves wrong.

> **Principle.** Always store sub-scores, never only the composite. `{ structural: 0.91, type: 0.95, name: 0.61 }` explains a match; `0.87` only asserts it.

7.3 Provenance: The `basis` Field

Beyond **how** confident, the set records **where each correspondence came from**. A `basis` field tags the origin:

basis	Meaning
`fk-reachable`	derived from a foreign-key path in the schema graph — high confidence, deterministic
`name-heuristic`	proposed by name similarity among structurally plausible candidates — review
`ai-inferred`	supplied by a language model for a gap deterministic rules could not close
`user-confirmed`	accepted or corrected by a human — locked

Provenance is what makes human review **targeted** rather than exhaustive. A reviewer need not re-examine every foreign-key-reachable correspondence; those are structural facts. Attention goes to the `name-heuristic` and `ai-inferred` rows — the correspondences that rest on the fallible signals — which is the highest-value use of a reviewer's time. The set does not hide its uncertainty in an average; it localises it in a field.

7.4 Function and Status: The Completed Entry

A correspondence is not finished when it has a source and a score. From Chapter 6 we know that some target fields are projections and others are labelled nulls — derivations the tgds cannot fill. The entry therefore also carries the **mapping class** (the shape of the derivation — direct, array-transform, lookup-join, aggregate, and so on), the **inferred function** where one can be named, and an **mc-status**: `complete` when the field is fully accounted for, `derivation-gap` when it needs computation, `ambiguous` when several candidates compete, `unmatched` when no input can supply it.

This is what lifts the correspondence set from a coverage report to a **structural draft of the transformation**. A reviewer reading it does not see "83% covered"; they see, field by field, that this one is a direct copy, that one a `map`, that one a `findBy` against the catalogue, and that one a `sum` that no input provides and a human must confirm. The next two chapters are about computing those last columns; the set is where they are recorded.

7.5 The Serialized Form

The set is persisted as JSON — git-versioned, diffable, dependency-free. Against the alternatives (ontology-alignment formats, tool-internal XML, raw dependency notation), plain JSON wins on exactly the properties that matter for an artifact that lives beside source code: score changes between schema versions show up in a diff; sub-scores and `basis` travel with each entry; and no external toolchain is required to read it.

Two structural features handle the N-input case directly. Each correspondence names its `src_input` and a `tgd_class` — `primary`, `enrichment`, or `lookup` — so the three kinds of dependency from Chapter 6 remain distinguishable in the data. And the name–value inputs do not appear among the correspondences at all; they live in a separate `bindings` array, each a direct value injection with a key and no score fields, because a constant has no structural confidence to report. The shape of the artifact mirrors the shape of the theory: scored correspondences for the structurally-rich inputs, unscored bindings for the constants.

7.6 One Artifact, Two Consumers

The correspondence set is deliberately **dual-purpose**, and recognising this shapes its design. It is a persistence format — the durable, reviewable record of a mapping's analysis. It is also an **interchange** format: the structured context handed to a language model when generation is needed. An AI asked to write the transformation does not receive raw schema pairs and an open instruction; it receives pre-classified inputs, pre-computed correspondences with their mapping classes, and explicitly flagged derivation gaps. Its task is thereby **scoped**: fill the gaps, resolve the ambiguities, and leave the structural skeleton alone. Structured context with targeted gaps is a fundamentally stronger prompt than an open-ended schema pair, and the correspondence set is what makes that prompt possible.

The same artifact also drives the visual mapper: each entry becomes a connector with a function badge, a status colour, and a confidence shade keyed to its `basis`. Persistence, machine interchange, and human visualisation are three readings of one object — which is the strongest possible argument that the analysis of Part II was worth formalising. The set is where the theory becomes something you can hold.

Part III

From Theory to Transformation

Typing inputs, analysing outputs, inferring functions, and strategy-first generation

8 Typing the Inputs

Part II reasoned as though every input were a schema graph waiting to be matched. The inputs of an N:1 mapping are not so uniform. Before any matching can sensibly begin, each input must be **classified** — by the role it plays and by the structure it has — because the type decides whether an input enters the matching pipeline at all, and how. This chapter develops input typing, and grounds it in a body of practice older than schema matching: the classification of enterprise data itself.

8.1 Why Type Before You Match

The argument is short and decisive. Running a structural matcher over a bag of configuration values is not merely wasteful; it is meaningless. A configuration set has no entities, no keys, no foreign keys — nothing for a graph matcher to align. Feeding it to the matcher produces noise, and noise that looks like signal is worse than silence. Conversely, a rich payload and a rich lookup **must** go through the full graph treatment, and treating them as interchangeable misses the join that defines their relationship.

So typing is a **gate**. It routes each input to the treatment its kind deserves: structurally-rich inputs into graph construction and matching, name–value inputs straight to bindings, schema libraries into shared-type resolution, registries into entity extraction. Get the gate wrong and every downstream stage inherits the error; get it right and each stage receives exactly the inputs it can use.

8.2 The Role Taxonomy

An input's **role** is one of a small set:

Type	Role
`payload`	primary message contract — rich structure, keys, nesting
`chain`	a payload from a prior pipeline step — same structural class, temporal role
`lookup`	reference / enrichment data — structurally rich, but joined rather than driven
`nvp`	name–value pairs — flat, homogeneous; drives variable bindings
`config`	an nvp subtype — deployment constants, candidates for compile-time inlining
`enumeration`	a closed value set — drives validation, not field binding
`shared-type-library`	an import dependency — provides types to other schemas, no standalone role
`meta-schema`	a schema registry — requires entity extraction before use

What is striking is that this taxonomy is not invented. It is, almost line for line, the classification that enterprise data management has used for decades.

8.3 Grounding: Master-Data Classification

Master-data management divides enterprise data into a handful of kinds: **master data** (the durable business entities — customers, products), **transactional data** (the events — orders, invoices), **reference data** (the shared code lists), **metadata** (data about data), and **configuration** (the environment). Lay the role taxonomy against it and the correspondence is nearly exact:

Role	Master-data class
`payload`, `chain`	transactional data
`lookup`	master / reference data
`enumeration`	reference data (code list)
`config`, `nvp`	configuration
`meta-schema`	metadata (a registry)

This is the strongest possible evidence that the taxonomy is well-grounded: it was arrived at from the mechanics of mapping, and it lands on the categories an entirely separate discipline reached from the mechanics of data governance. The two converge because they are describing the same thing — the kinds of role data plays in an enterprise — from two directions.

The `meta-schema` row deserves a note in the vocabulary of Chapter 2. If a schema is **metadata** — data about a class of data — then a registry of schemas is **meta-metadata**, sitting one rung higher on the ladder. A `meta-schema` input is therefore not data to map but metadata to **unpack**: its entities must be extracted before any of them can be typed as a payload or lookup in their own right.

8.4 The Weak Seam: `config` Versus `nvp`

The grounding also exposes the taxonomy's softest joint. Master-data classification puts `config` and `nvp` in one box — configuration — and structurally they are indistinguishable: both are flat maps of keys to scalar values. Any attempt to separate them by **shape** (homogeneous strings on one side, mixed scalar types on the other) is a thin and unreliable test, because the real difference is not structural at all. It is role and lifecycle — and, in the three-times vocabulary of Chapter 3, it is precisely the init/run boundary: `config` is bound once at **init time** and may be inlined; `nvp` is read per message at **run time**. No inspection of shape can recover that distinction reliably, because the difference is one of **time**, not structure.

The lesson generalises into a rule: **do not infer what should be declared.** The split between `config` and `nvp` is a property the platform or the developer knows and the schema does not encode. The typing engine should infer a single "binding" kind from structure and refine to `config` or `nvp` only from a declaration. Inference is for facts the structure carries; declaration is for facts it cannot.

8.5 The Priority Cascade

Declared knowledge, where it exists, must win. The typing engine resolves an input's type through a strict cascade:

Priority	Source of the type
1 — highest	platform declaration (a fixed pipeline slot) — no inference run
2	an inline schema tag — user-declared, inference skipped
3	a transformation-header declaration — user-declared, inference skipped
4	structural classification tests — inference, the fallback
5 — lowest	unresolvable — surface an error rather than guess

The inference tests at level 4 are the fallback, not the first resort. They exist for the case where nothing is declared: a schema arrives with no platform slot, no inline tag, no header. Then, and only then, structural observation decides — nesting depth, the presence of keys, the document root, the namespace relationship. And when even that cannot decide — a set of peer schemas with no entry point, say — the right answer is an **error**, not a guess. A typing engine that guesses under ambiguity manufactures false confidence; one that declines forces the ambiguity to the surface where a human can resolve it.

8.6 Two Levels of Typing

Typing the input as a whole is only half the story, and the smaller half. The granular signal — the one that predicts mapping strategy — lives a level down, **inside** the schema.

Level	What it types
schema-level	the input's role: `payload`, `lookup`, `nvp`, …
node / graph-level	the structure within: entity, key, reference (foreign key), containment, value

The header–detail relationship of Chapter 4 — the order that contains and is referenced by its lines — is a node-level fact. It is not a new kind of input; the input is a payload. It is a relationship **inside** the payload graph, and it is what tells the mapper to reach for `map` and `groupBy`. A theory that types only whole inputs is blind to it. Typing must reach both levels: the role of the input, and the structure of its insides.

8.7 The Structural Archetype

There is a useful third axis, orthogonal to role, that names the **shape** of a structurally-rich input:

Archetype	**Graph signature**	**Predicts**
single-record	one entity, no repeating groups	a field copy or spread
flat-list	one repeating entity, no nesting	`map`
header-detail	a parent entity with a child collection by containment or foreign key	`map` + `groupBy`
star / snowflake	a fact entity with foreign keys to dimension lookups	`findBy` / `filterBy` joins
deeply-nested	depth three or more, mixed cardinalities	composed `map` and spread

This is the "kind of message" intuition made explicit, and it has its own pedigree: the fact-and-dimension vocabulary of dimensional modelling, and the weak-entity vocabulary of entity-relationship modelling. An IDOC order is a header-detail message; a transaction joined to reference tables is a star. The archetype is derived from graph topology, it is orthogonal to the input's role, and — crucially — it **predicts the strategy** before a single correspondence is scored. It is the structural cousin of the design patterns a schema serializer already tracks, lifted from serialization into mapping.

8.8 Where the Type Lives

A final, architectural point. An input's type — its role, its archetype, its resolution provenance — is a property of the **schema**, decided at design time. It must be recorded where Chapter 2′s discipline says schema facts belong: on the schema document, in a metadata envelope around the schema graph, never smeared across the data nodes of an instance. A datum in a parsed order has no "payload versus lookup" identity; that is the role of the input **slot**, not of any value flowing through it.

Recording the type on a new schema-document envelope — distinct from the runtime data model — has a decisive practical consequence: it touches nothing on the execution hot path. The data model the transformation produces is unchanged; the parsers, the engine, the serializers, the conformance suite all see exactly what they saw before. The type is additive, design-time, and carried by an object that did not exist until the analysis needed it. Typing the inputs costs the run-time nothing, which is precisely what Chapter 3′s separation of modes demands.

9 Output Analysis

Typing the inputs has a dual that is easy to overlook, and overlooking it is the most common way an N:1 analysis lies. The output contract is not a passive target that correspondences merely point at; it is an object to be **analysed in its own right**. Each output field has an intrinsic kind — copied, calculated, looked-up, aggregated, generated — and much of that kind can be read from the output schema **before any input is consulted**. This chapter develops target-field analysis and grounds it in the literature of data provenance.

9.1 The False-Coverage Problem

Begin with a report that looks reassuring and is wrong:

> `source: order.lines[*].price` → `target: invoice.orderTotal` — **matched**.

This is not a match. The order total is not the price of a line; it is the sum, over all lines, of quantity times price. The report has dressed a **derivation gap** as a correspondence, and in doing so has told two lies: it has claimed a field is sourced when it must be computed, and it has inflated the coverage percentage with a field that is not, in fact, covered. A mapping analysis that does this gives false confidence exactly where confidence is most expensive.

The same failure has a quieter form. `order.lines[*]` → `invoice.lineItems[*]` is a genuine correspondence, but reporting it as "matched" omits that it **implies a `map`** — and, if the target is ordered, an `orderBy` before the `map`. That the lines become line items is a schema-level fact; that the transformation needs a `map` to make it so is also a schema-level fact. It belongs in the analysis, not in the reader's head.

The root cause is the value invention of Chapter 6. A target field that is a labelled null — a value that must exist but is copied from nothing — cannot be honestly reported as a match. It must be reported as what it is: a field that needs computation, lookup, generation, or a constant.

9.2 A Taxonomy of Target Fields

Give each output field a kind, read mostly from the output schema's own names, types, constraints, and structural context:

Kind	Signal (mostly schema-local)	Realised by
copy / direct	a scalar with a type-compatible source	a field reference
calculated / derived	a composite or formatted field (`fullName`, `displayDate`)	`concat`, formatting, arithmetic
aggregate	a numeric over a sibling repeating group (`total`, `count`)	`sumBy`, `reduce`, `countBy`
lookup-required	a descriptive field beside a code, with a lookup input present	`findBy`, `filterBy`
constant / literal	required, no candidate, fixed or enumerated	a literal, or a config binding
generated / system	`id`, `uuid`, `timestamp`, `correlationId`	a generator, or a pipeline-var binding
conditional	a flag or status that depends on a condition	`if`, `match`
default / optional	optional, with a default	omission or a default
unmapped	required, with no plausible source or derivation	— a true gap

The value of this taxonomy is that it is **target-driven**. A numeric field named `total` sitting beside a collection of lines all but announces itself as an aggregate, with no input examined at all. A descriptive field named `productName` beside a `productCode`, in a mapping that has a lookup input, announces itself as a join. The output schema is not silent about how its fields must be produced; it is, read carefully, remarkably talkative.

9.3 Schema-Local Versus Source-Dependent

How much of a field's kind is decidable from the output alone, and how much waits on the inputs? The division is clean.

The **kind** is largely schema-local: name, type, constraint, and sibling cardinality decide whether a field is a copy-candidate, an aggregate, a lookup, a constant, or a generated value. What the source decides is narrower: whether a copy-candidate actually **has** a source (copy) or does not (gap); whether the lookup input the field wants is actually present. The output analysis runs first, at contract-load, with no inputs; alignment then resolves the open question of source availability. The composition is exact:

$$\text{mapping class} = \text{output-field kind} \times \text{source availability}$$

The mapping class of Chapter 7 — the entry that says "this is a `map`, that is a `findBy`, this is a `sum` no input provides" — is the **product** of the target-field analysis and the source-side correspondences. The `derivation-gap` that Chapter 6 first named is simply the cell where the output-field kind is "aggregate" or "calculated" and no projection can fill it. Output analysis is the target-driven half of function inference; the next chapter is the synthesis.

A small but genuine role remains for AI here. Semantic field names — does `netDue` mean an aggregate? does `clientRef` mean a lookup? — sometimes exceed what name-and-type rules can decide, and a language model can classify them where deterministic rules cannot. Such classifications are marked, as always, by their basis, so a reviewer knows which target kinds were read from structure and which were guessed.

9.4 Grounding: Provenance, ETL, and Synthesis

The question "how is this output value produced?" is not new. Four bodies of work answer it, and together they ground the taxonomy.

Data provenance and lineage is the formal study of exactly this. How-provenance — the semiring account of how an output value is computed from inputs — and the lineage-tracing literature classify transformations by the output-to-input relationship: a value dispatched unchanged, a value aggregated, a value produced by an opaque computation. The copy / aggregate / calculated distinction is the provenance distinction, recovered from the mapping side.

ETL practice is the taxonomy's nearest practical cousin. Decades of data-warehouse engineering classify target columns into precisely these kinds — direct, derived-computed, surrogate-key (generated), lookup-decode, default — and the functoid and component catalogues of mapping tools (constant, autonumber, lookup, aggregate, arithmetic, string, date) are target-field-derivation taxonomies in everything but name. The output taxonomy is what those catalogues have always implied.

Data-exchange value invention supplies the formal boundary, already met in Chapter 6: the labelled null is the copy-versus-derive line, and the derived kinds — calculated, aggregate, generated — are exactly the target values that data-exchange theory declares beyond a tgd.

Program synthesis is the natural successor. Once a field is typed `calculated`, the remaining task is to **synthesise its expression**, and programming-by-example — generating a transformation from input–output samples — is the literature for that step. Typing the field is the analysis; synthesising its formula is the generation.

9.5 Synthesising a Sample Output

Output analysis also answers a practical temptation flagged back in Chapter 3. A developer often wants a **sample** output document — something concrete to inspect. The wrong way to get one is to switch to Execution mode, run the transform on a sample input, and capture the result; that quietly crosses the mode boundary and obtains the output by execution.

The right way stays in Message Contract mode: **synthesise** a representative instance from the output **contract**. The schema describes the shape; a generator can populate it with representative values. This produces an output sample with no transformation run at all — schema-driven synthesis, not execution. It is the output-side mirror of the principle that has governed the whole book: in Message Contract mode, reason from shape, and let data be how you check the result later, in the other mode.

9.6 What Output Analysis Buys

Folding output analysis into the Message Contract pass does not make the report more complex to read; it makes it more useful. Without it, a developer receives a list of field matches and must, for every field, re-derive in their head whether it is a copy, a sum, or a join — re-doing by hand the analysis the tool declined to do. With it, the report is a **structural draft of the transformation**: each output field already labelled with how it must be produced. The developer corrects and refines a draft rather than starting from a blank page. That is the efficiency the whole pipeline is ultimately after, and it begins by taking the output as seriously as the input.

10 Function Inference

To know that an output field maps from a source is half of an answer. The other half is **which UTL-X construct** realises the mapping — a field reference, a `map`, a `findBy`, a `sum`. Function inference supplies that half, and the argument of this chapter is that it is not a separate layer bolted onto the analysis but the **completion** of it. A correspondence without an inferred function is an unfinished Message Contract result. The schema graph, read correctly, names most of these functions on its own; where it cannot, it says so.

10.1 The Mapping Class Is the Unit

The carrier of the inference is the **mapping class** introduced in Chapter 7. Each correspondence is assigned one, and each class is bound to a UTL-X construct and a completion status:

Mapping class	UTL-X	Status
direct	field reference	complete
array-transform	`map` / `mapBy`	complete
lookup-join	`findBy` / `filterBy`	complete
spread-merge	`...spread`	complete
grouped-transform	`groupBy` + `mapBy`	complete
sorted-transform	`orderBy` + `map`	complete
derivation-gap	`sumBy` / `reduce` / arithmetic	incomplete — needs computation
unmatched	—	incomplete — needs a human
ambiguous	—	incomplete — needs review

The mapping class is the synthesis promised by the last chapter: the product of the output-field kind (target-driven) and the source availability (from alignment). An output-field kind of "aggregate" with a source collection present yields `grouped-transform` or a `derivation-gap`, depending on whether a grouping key exists; a kind of "lookup-required" with the lookup input present yields `lookup-join`; a kind of "copy" with a type-compatible source yields `direct`. The class is where the two halves of the analysis meet.

10.2 Inference From Graph Signals

The remarkable thing — and the reason inference belongs **inside** Message Contract mode rather than in some later AI stage — is how much of it is **deterministic from the schema graph**. The graph's structural signals map almost directly onto function classes:

Graph signal	Inferred function
a `lookup`-typed input reached by a foreign-key edge	`findBy` / `filterBy`
a one-to-many containment, source to target	`map` / `mapBy`
an N-to-M junction entity	`groupBy` + `mapBy`
a one-to-many collapsing to a one-to-one scalar	aggregation — a derivation gap
several inputs into one target object	`...spread`
a one-to-many feeding an **ordered** one-to-many	`orderBy` + `map`
a required target field with no source and a numeric context	a derivation gap — computation
a required target field with no source and no numeric context	unmatched — flag for a human

None of these is a guess. A foreign-key edge into a lookup **is** a join; a one-to-many containment **is** a `map`. These are structural facts of the graph, read off it the way Chapter 4 read off the header–detail pattern. The schema typology of Chapter 8 and the graph of Chapter 4 together have strong predictive

power over which construct each correspondence needs — strong enough that the deterministic core of the analysis can name the function for the great majority of fields without consulting an oracle.

10.3 The tgd Boundary, Revisited

Function inference is also where the boundary of Chapter 6 becomes operational. The functions above the line in that chapter's table — references, spreads, maps, keyed lookups — are tgd-expressible, and the graph derives them deterministically. The functions below the line — `groupBy`, `sumBy`, `reduce`, derived arithmetic, `orderBy` — are **not** tgd-expressible; they compute or reorder rather than move and reshape. For these, inference can identify **that** a computation is needed and **what kind** (a sum over this collection, a count of those rows) but cannot, from the graph alone, author the exact expression. They are marked `derivation-gap`, with a description of what is required, and handed onward — to a human, or to the program-synthesis step of Chapter 9′s closing.

This is the honest version of the analysis. It does not pretend the graph determines everything; it determines the skeleton, and it labels precisely where the skeleton ends and computation begins. A `derivation-gap` is not a failure of inference — it is inference correctly reporting the limit of what structure can decide.

10.4 The Deterministic / AI Split

The full pipeline is a hybrid, and the value of function inference is that it keeps the hybrid **clean**: deterministic and AI work are separated, not blended, with AI invoked as a targeted enrichment at specific, named points rather than as a general overlay.

Phase	**Deterministic or AI**
schema typing	mostly deterministic; AI only at the chain/payload boundary
graph construction	fully deterministic — pure parsing of declared constraints
structural alignment	mostly deterministic; AI for implicit foreign keys, choice groups, cardinality intent
name refinement	AI-primary — the semantic bridging no metric achieves

Graph construction must remain fully deterministic: it is the stable foundation everything else depends on, and there is nothing in parsing declared constraints for an oracle to improve. Structural alignment is mostly deterministic, with AI earning its place only at the genuine ambiguities — an implicit foreign key with no declared `keyref`, an `xs:choice` whose intended branch is a semantic judgement, a cardinality mismatch that might be a deliberate flattening or an error. Name refinement is where AI leads, because it is the one phase whose core problem — bridging `customerId` to `clientRef`, or one language to another — is irreducibly semantic.

Every inference, deterministic or otherwise, records its basis. A function derived from a foreign-key edge is marked `fk-reachable`; one proposed by a model for an implicit key is marked `ai-inferred`. The provenance discipline of Chapter 7 thus extends to the inferred functions themselves: a reviewer can see not only which construct was chosen but on what authority, and can spend their scrutiny on the constructs that rest on inference rather than on structural fact.

10.5 Completing the Message Contract Report

The chapter's thesis — that function inference completes the Message Contract analysis rather than following it — has a concrete consequence for what the report **is**. Without inference, a Message Contract report is a coverage percentage and a list of matched fields: a developer reading it must still decide, field by field, what construct each match implies, re-deriving the analysis by hand. With inference, the report is a **first draft of the transformation's structure**: every output field carries not only its source but the construct that realises it, and every gap is labelled with the computation it awaits.

That is the efficiency argument for folding inference into Message Contract mode rather than treating it as a separate stage. The developer no longer translates a coverage report into a transformation; they correct and refine a structural draft. The analysis has done the part that is mechanical — the skeleton the graph determines — and reserved human attention for the part that is genuinely a decision. Generating the transformation from that draft, strategy-first, is the final step, and the subject of the last chapter.

11 Strategy-First Generation

Everything in this book has been preparation for one act: producing a transformation. The analysis — typed inputs, a schema graph, a scored correspondence set, inferred functions — is not an end in itself. It exists so that the transformation that finally gets written is not merely correct but **idiomatic**: a mapping that reaches for spread, `map`, joins, and `groupBy` because the analysis told it to, rather than one that fills two hundred fields one at a time. This closing chapter assembles the pipeline that does that, and then steps back to ask what the whole theory was for.

11.1 Why One Fixed Prompt Fails

Imagine generating an N:1 mapping with a single instruction: "fill every output field from a source." It is the obvious approach, and it fails in three ways that this book has spent its chapters diagnosing.

It **enumerates**. Asked to fill each field, a generator writes an assignment per field — two hundred lines where `...$payload` and a handful of overrides would do. The structure of the instruction bakes in per-leaf copying, and the result is verbose where it should be terse.

It **ignores the language**. Spread, pipelines, `map`, `groupBy`, `reduce`, joins — the expressive constructs that make UTL-X worth using — are rarely produced, because the task has been framed as field-filling and field-filling has no place for them.

It **flattens the inputs**. "A source" treats every input as an equal candidate, and so misses that the second input is a lookup joined by key, not a co-equal place to find fields. The join shape — the very thing Chapter 4 worked to recover — never appears.

These are the three failures of Chapter 1 — shape, expressiveness, asymmetry — returning as failures of **generation**. A single prompt cannot avoid them because no single prompt encodes every mapping shape.

11.2 The Planning Layer

The fix is to insert a planning layer before generation, turning a one-step process into a pipeline:

```
mode → analyse → strategy → strategy-specific generation
```

A first pass analyses the problem and **chooses a strategy** — a shape for the mapping as a whole. Generation then proceeds with the prompt and the scaffold that fit that strategy, so it reaches for the right constructs by construction. The division of labour is the one the book has built toward: **strategy decides the shape; the correspondence set decides the sources.** A mirror mapping and a join mapping are generated differently not because the model is asked nicely but because they are different problems, recognised as such before a line is written.

11.3 The Strategy Catalogue

The strategies are few, and each carries its own generation shape and its own scaffold:

Strategy	When	Shape
mirror / pass-through	the output closely follows one input	`...$driver` plus overrides — never enumerate
restructure	the same data in a new shape	explicit object construction
array-transform	the driver is a collection	`map` over the driver
join / lookup	a driver plus reference tables	index the lookup, look up per row
aggregate / group	many rows collapse to few	`groupBy` / `reduce`

The decisive detail is that the **scaffold** is per-strategy. This is what resolves the enumeration problem at its root: a mirror mapping is seeded with a spread, a restructure with field holes, a join with the index-

and-lookup skeleton. The generator is not asked to remember not to enumerate; it is handed a starting point in which enumeration is not the shape. Real mappings compose these — `map` the driver, spread its fields, look up the reference per row, default the gaps — but each strategy contributes a recognisable, idiomatic fragment rather than a wall of assignments.

11.4 Deterministic First, AI Where It Earns Its Place

The analysis that chooses the strategy is **deterministic-first**. Cardinalities, key detection, the mirror ratio, the structural archetype — these come from the schema graph at almost no cost, and they are enough to select a strategy in the common case. A small, targeted call to a language model is reserved for genuine ambiguity: a tie between two strategies, an implicit key, a semantic name gap. This inverts the cost profile of the single-prompt approach, where one large generative call did everything slowly and opaquely. Here the bulk of the work is cheap, fast, and explainable, and the oracle is consulted only where structure genuinely runs out.

Self-correction — validating the generated transformation against the engine and repairing it — remains, but as a **safety net**, not the primary mechanism. A pipeline that relies on generate-then-fix as its main engine has admitted it does not understand the problem; one that analyses, selects, and generates, and only then validates, uses correction to catch the residue rather than to substitute for understanding.

11.5 Validate as the Trigger

All of this is Message Contract mode, and it has a natural home in the tooling. In Message Contract mode the primary action is not **execute** — there is no data to execute on — but **validate**. So the Validate action is exactly the right trigger for the whole pipeline: it runs the analysis, with no execution, and surfaces its stages — classify the inputs, find the correspondences, refine the uncertain ones, report coverage. Where Execution mode runs the data, Validate runs the **analysis**. The symmetry is clean: two actions, two modes, two questions — what does this produce, and is this mapping sound.

11.6 The Visual Surface

The correspondence set, the typed inputs, and the inferred functions are not only fuel for a generator; they are a **picture**. Rendered, they become a mapper in the familiar idiom: the input schemas as trees on one side, the output contract as a tree on the other, and between them the derivations — a direct copy as a plain connector, a function as a labelled block, a gap as a flagged node. Each connector's colour carries its status and its basis, so a structurally certain copy and an AI-inferred guess do not look alike. The same analysis that grounds the generated code grounds the diagram a human reads; persistence, generation, and visualisation are three uses of one object, which has been the recurring sign throughout this book that the formalisation was worth it.

11.7 From Many, One

It is worth stating plainly what has been built, because it is easy to lose in the machinery. An N:1 mapping began, in Chapter 1, as a craft: several inputs, one contract, and a developer's judgement filling the space between. The chapters since have turned that craft into something that can be **reasoned about**. The inputs and output share one model. Schemas are graphs. A mapping is a formal object — a set of dependencies, with a clean line between the structure a graph determines and the computation it cannot. The analysis is a scored, provenance-bearing artifact. The inputs are typed by role and structure; the outputs by derivation. The functions are inferred where structure decides and flagged where it does not. And the transformation is generated strategy-first, idiomatic by construction.

The result is not that mappings get written for you — judgement remains, especially at the derivation gaps and the semantic name bridges where the theory honestly says structure runs out. The result is that you can say, precisely, what a mapping **does**: which fields are copied and which derived, which inputs are sources and which are looked up, where it is complete and where a human must still decide. That is the

difference between writing an N:1 mapping and **understanding** one — and it is why, for the architect who wants the second, the theory was worth the book.

From many inputs, one output. From a craft, a discipline. From many, one.

Part IV

The Classical Foundation

The theory of 1:1 mapping, and an annotated bibliography

12 The Classical Theory of 1:1 Mapping

This book has been about **many** inputs becoming one output. The literature it draws on was, almost without exception, written about **one** source and one target. That is not a defect to apologise for; it is the foundation to build on. The classical theory of 1:1 mapping — how a single source schema relates to a single target schema — is mature, peer-reviewed, and, as Chapter 6 showed, generalises to N:1 by the simple act of letting the source be a union. This closing chapter narrates that foundation and annotates its canon, so that the references scattered through the preceding chapters stand together as a reading list.

Each entry below gives a citation, a short abstract, and a one-line *Brings* — the single thing the work contributes to this book's argument.

12.1 The 1:1 Assumption, and Its Quiet Generalisation

Read the schema-matching and schema-mapping literature and you will find an implicit picture: a source schema on the left, a target schema on the right, correspondences drawn between them. The matchers assume a source/target **pair**. The mapping generators discover a query from one schema to another. The picture is so consistent that the 1:1-ness of it goes unstated.

Yet, as Chapter 6 observed, the data-exchange formalism never required it. A schema mapping $M = (S, T, \Sigma)$ places no constraint that S be a single schema; in the relational setting S is already a set of relations, and the dependencies in Σ range over all of them. The N:1 problem of this book is therefore not an extension of the theory but a reading of it that the diagrams happened to suppress. Everything below was developed for the 1:1 case and applies to N:1 once the source is understood as the union of the inputs — with the one caveat, also from Chapter 6, that name–value inputs are constants, not schemas, and sit outside the matching machinery entirely.

12.2 Schema Matching

The matching problem is: given two schemas, find the correspondences between their elements. It is the input to mapping, and its literature gives us the taxonomy, the signals, and the discipline of combining them.

Rahm, E. & Bernstein, P. A. (2001). "A Survey of Approaches to Automatic Schema Matching." *VLDB Journal* 10(4). — The definitive taxonomy. Classifies every matcher along orthogonal axes: schema-only versus instance-based, element-level versus structure-level, linguistic versus constraint-based, individual versus combining.
Brings — the taxonomy that becomes the book's architectural skeleton: the two-modes split, graph-before-names, and keys-over-names are its axes.

Madhavan, J., Bernstein, P. A. & Rahm, E. (2001). "Generic Schema Matching with Cupid." *VLDB.* — A structure-aware matcher that computes element similarity and propagates it up and down the schema tree, so agreement among children reinforces their parent and vice versa.
Brings — similarity propagation through structure — the case for matching with structural context rather than isolated names.

Do, H. H. & Rahm, E. (2002). "COMA — A System for Flexible Combination of Schema Matching Approaches." *VLDB.* — A composite matcher: a library of individual matchers whose scores are combined under configurable strategies and weights.
Brings — composite, weighted multi-signal scoring — the direct model for the sub-scored correspondence set of Chapter 7.

Melnik, S., Garcia-Molina, H. & Rahm, E. (2002). "Similarity Flooding: A Versatile Graph Matching Algorithm and its Application to Schema Matching." *ICDE.* — Casts matching as a fixpoint computation: similarity flows along the edges of a combined graph until it settles.
Brings — graph-propagation matching — the algorithmic view that maps onto UDM schema-graph alignment.

12.3 Schema Mapping and Data Exchange

Matching finds correspondences; **mapping** turns them into an executable relationship, and **data exchange** makes that relationship a formal object. This is the theory that lets us say what a mapping **is**.

Miller, R. J., Haas, L. M. & Hernández, M. A. (2000). "Schema Mapping as Query Discovery." *VLDB.* — The foundation of the Clio line. Reframes mapping as discovering a query from source to target, driven by value correspondences and schema structure rather than authored by hand.
Brings — mapping-as-discovery: a mapping is derived from structure, not hand-written.

Popa, L., Velegrakis, Y., Miller, R. J., Hernández, M. A. & Fagin, R. (2002). "Translating Web Data." *VLDB.* — Clio's mapping generation using the schemas' referential constraints to produce nested mappings, inventing target values via Skolem functions where the source supplies none.
Brings — joins and nesting from keys (not names), and value invention by Skolem function — the formal root of the lookup join and the derivation gap.

Haas, L. M., Hernández, M. A., Ho, H., Popa, L. & Roth, M. (2005). "Clio Grows Up: From Research Prototype to Industrial Tool." *SIGMOD.* — The account of Clio's maturation into a usable tool.
Brings — evidence that schema-mapping theory is engineering-ready, not a paper exercise.

Fagin, R., Haas, L. M., Hernández, M., Miller, R. J., Popa, L. & Velegrakis, Y. (2009). "Clio: Schema Mapping Creation and Data Exchange." In *Conceptual Modeling: Foundations and Applications*, Springer. — A retrospective tying mapping generation to data-exchange semantics.
Brings — the consolidated bridge between "how mappings are generated" and "what a mapping formally is."

Fagin, R., Kolaitis, P. G., Miller, R. J. & Popa, L. (2005). "Data Exchange: Semantics and Query Answering." *Theoretical Computer Science* 336(1) (originally ICDT 2003). — Defines a schema mapping formally as a set of source-to-target tuple-generating dependencies, and develops universal solutions, the chase, and certain answers.
Brings — the formal object itself: tgds, $M = (S, T, \Sigma)$, and the boundary between structural reshaping and computation.

Bellahsene, Z., Bonifati, A. & Rahm, E. (eds.) (2011). "Schema Matching and Mapping." Springer, *Data-Centric Systems and Applications.* — A single volume surveying the whole field above.
Brings — the one-stop survey: the recommended entry point for an implementer.

12.4 Data Classification and Modeling

Mapping theory says how schemas relate; data-management theory says what **kinds** of data there are. This is the grounding for typing the inputs (Chapter 8) and naming the structural archetypes.

Chen, P. P. (1976). "The Entity-Relationship Model — Toward a Unified View of Data." *ACM TODS* 1(1). — The origin of entities, relationships, and cardinality, and of the **weak entity** under an identifying relationship.
Brings — the vocabulary for header–detail: an order line is a weak entity, identified only within its order.

Kimball, R. & Ross, M. "The Data Warehouse Toolkit." Wiley. — Dimensional modelling: the fact-versus-dimension distinction and the star and snowflake schemas.
Brings — the structural-archetype axis: a lookup is a dimension, a payload-with-lines is a fact.

Kimball, R. & Caserta, J. "The Data Warehouse ETL Toolkit." Wiley. — Classifies target columns into direct, derived-computed, surrogate-key, lookup-decode, and default.
Brings — a target-field-derivation taxonomy in all but name — the practitioner root of the output analysis of Chapter 9.

DAMA International. "DAMA-DMBOK: Data Management Body of Knowledge." Technics Publications. — The enterprise taxonomy of master, transactional, reference, metadata, and configuration data.
Brings — near-exact external validation of the input role taxonomy of Chapter 8.

Loshin, D. (2008). "Master Data Management." Morgan Kaufmann. — The standard treatment of master versus transactional versus reference data.
Brings — the grounding for `lookup`-as-master/reference and `payload`-as-transactional.

Dreibelbis, A., et al. (2008). "Enterprise Master Data Management: An SOA Approach to Managing Core Information." IBM Press. — An architectural view of the same classification in an integration context.
Brings — the integration-architecture reading of the data-class taxonomy.

12.5 Integration and Messaging

The N:1 shape is, before it is a theory, an integration reality. This is the literature of messages and the patterns that move them.

Hohpe, G. & Woolf, B. (2003). "Enterprise Integration Patterns." Addison-Wesley. — The pattern language of messaging, including the **message types** (Document, Command, Event) and the **Canonical Data Model**.
Brings — the integration-level statement of "one model in the middle" that UDM realises, plus a message-kind classification.

UN/EDIFACT; ANSI ASC X12; SAP IDoc. — The dominant electronic-data-interchange and enterprise-messaging structures, with their control/header/detail/status segment hierarchies.
Brings — real, standardised header–detail messages: the IDOC intra-document foreign-key example, and proof the predicted structure is ubiquitous.

12.6 Data Provenance and Lineage

The output analysis of Chapter 9 asks "how is this value produced?" That is the provenance question, and it has its own deep literature.

Green, T. J., Karvounarakis, G. & Tannen, V. (2007). "Provenance Semirings." *PODS.* — How-provenance: an algebra (commutative semirings) describing **how** an output value is computed from inputs, not merely whether it depends on them.
Brings — the formal algebra of copy versus derive versus aggregate.

Cui, Y., Widom, J. & Wiener, J. L. (2000). "Tracing the Lineage of View Data in a Warehousing Environment." *ACM TODS* 25(2). — Lineage tracing: identifying the source tuples responsible for a derived result.
Brings — the warehousing root of target-field provenance — which sources produced a value.

Cui, Y. & Widom, J. (2003). "Lineage Tracing for General Data Warehouse Transformations." *VLDB Journal* 12(1). — Classifies transformations into dispatchers, aggregators, and black boxes by their output-to-input relationship.
Brings — a transformation-kind classification the target-field taxonomy mirrors.

Buneman, P., Khanna, S. & Tan, W. C. (2001). "Why and Where: A Characterization of Data Provenance." *ICDT.* — Distinguishes **why**-provenance (which inputs justified an output) from **where**-provenance (which input a value was copied from).
Brings — the conceptual scaffolding for asking, of each output field, where it came from.

12.7 Program Synthesis

Once a field is typed as calculated, the remaining task is to synthesise its expression. This is the literature of generating programs from examples.

Gulwani, S. (2011). "Automating String Processing in Spreadsheets Using Input-Output Examples." *POPL.* — The FlashFill work: synthesising a string transformation from a few input–output pairs. *Brings* — programming-by-example — the natural successor to a `derivation-gap`, where analysis names the need and synthesis supplies the formula.

12.8 Reading Order

For a newcomer to the theory, a path through the canon: begin with Rahm and Bernstein (2001) for the map of the territory; read Fagin et al. (2005) for the formal object; read the Clio trilogy (Miller et al. 2000; Popa et al. 2002; Fagin et al. 2009) for how mappings are generated from structure; and consult Bellahsene, Bonifati and Rahm (2011) when a single reference is wanted. The data-management works (Chen; Kimball; DAMA-DMBOK) supply the typing vocabulary; the provenance works (Green et al.; Cui and Widom) supply the output vocabulary. The rest of this book is, in a sense, an extended argument that these 1:1 foundations, taken together and read without the 1:1 assumption, are exactly what an N:1 mapping needs.

Bibliography

A plain, alphabetical list of the works cited. Annotated entries — with an abstract and a one-line contribution for each — appear in the preceding chapter, *The Classical Theory of 1:1 Mapping.*

Bellahsene, Z., Bonifati, A. & Rahm, E. (eds.) (2011). *Schema Matching and Mapping.* Springer, Data-Centric Systems and Applications.

Buneman, P., Khanna, S. & Tan, W. C. (2001). Why and Where: A Characterization of Data Provenance. *ICDT.*

Chen, P. P. (1976). The Entity-Relationship Model — Toward a Unified View of Data. *ACM Transactions on Database Systems* 1(1).

Cui, Y. & Widom, J. (2003). Lineage Tracing for General Data Warehouse Transformations. *VLDB Journal* 12(1).

Cui, Y., Widom, J. & Wiener, J. L. (2000). Tracing the Lineage of View Data in a Warehousing Environment. *ACM Transactions on Database Systems* 25(2).

DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge.* Technics Publications.

Do, H. H. & Rahm, E. (2002). COMA — A System for Flexible Combination of Schema Matching Approaches. *VLDB.*

Dreibelbis, A., Hechler, E., Milman, I., Oberhofer, M., van Run, P. & Wolfson, D. (2008). *Enterprise Master Data Management: An SOA Approach to Managing Core Information.* IBM Press.

Fagin, R., Haas, L. M., Hernández, M., Miller, R. J., Popa, L. & Velegrakis, Y. (2009). Clio: Schema Mapping Creation and Data Exchange. In *Conceptual Modeling: Foundations and Applications.* Springer.

Fagin, R., Kolaitis, P. G., Miller, R. J. & Popa, L. (2005). Data Exchange: Semantics and Query Answering. *Theoretical Computer Science* 336(1). (Originally ICDT 2003.)

Green, T. J., Karvounarakis, G. & Tannen, V. (2007). Provenance Semirings. *PODS.*

Gulwani, S. (2011). Automating String Processing in Spreadsheets Using Input-Output Examples. *POPL.*

Haas, L. M., Hernández, M. A., Ho, H., Popa, L. & Roth, M. (2005). Clio Grows Up: From Research Prototype to Industrial Tool. *SIGMOD.*

Hohpe, G. & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions.* Addison-Wesley.

Kimball, R. & Caserta, J. (2004). *The Data Warehouse ETL Toolkit.* Wiley.

Kimball, R. & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.

Loshin, D. (2008). *Master Data Management.* Morgan Kaufmann.

Madhavan, J., Bernstein, P. A. & Rahm, E. (2001). Generic Schema Matching with Cupid. *VLDB.*

Melnik, S., Garcia-Molina, H. & Rahm, E. (2002). Similarity Flooding: A Versatile Graph Matching Algorithm and its Application to Schema Matching. *ICDE.*

Miller, R. J., Haas, L. M. & Hernández, M. A. (2000). Schema Mapping as Query Discovery. *VLDB.*

Popa, L., Velegrakis, Y., Miller, R. J., Hernández, M. A. & Fagin, R. (2002). Translating Web Data. *VLDB.*

Rahm, E. & Bernstein, P. A. (2001). A Survey of Approaches to Automatic Schema Matching. *VLDB Journal* 10(4).

UN/EDIFACT (United Nations/Electronic Data Interchange for Administration, Commerce and Transport); ANSI ASC X12; SAP IDoc (Intermediate Document). Electronic-data-interchange and enterprise-messaging structure standards.

From Many, One

Every integration eventually faces the same shape: several inputs — a payload, a prior pipeline step, a lookup table, a handful of configuration values — must become **one** output that satisfies a fixed contract. Writing that transformation is a craft. **Reasoning** about it is a theory.

This book develops that theory. It treats a mapping not as a pile of field assignments but as a formal object: a schema graph, a set of source-to-target dependencies, a scored correspondence set. It draws the line — sharply — between Execution mode, which transforms real data, and Message Contract mode, which reasons about schemas before a single byte flows. And it grounds everything in the Universal Data Model, the one representation that makes "all formats" mean something.

A companion to **UTL-X: One Language, All Formats** — for the architect who wants to understand **why** an N:1 mapping is correct, not just that it runs.

GLOMIDCO B.V. · The Netherlands · 2026